



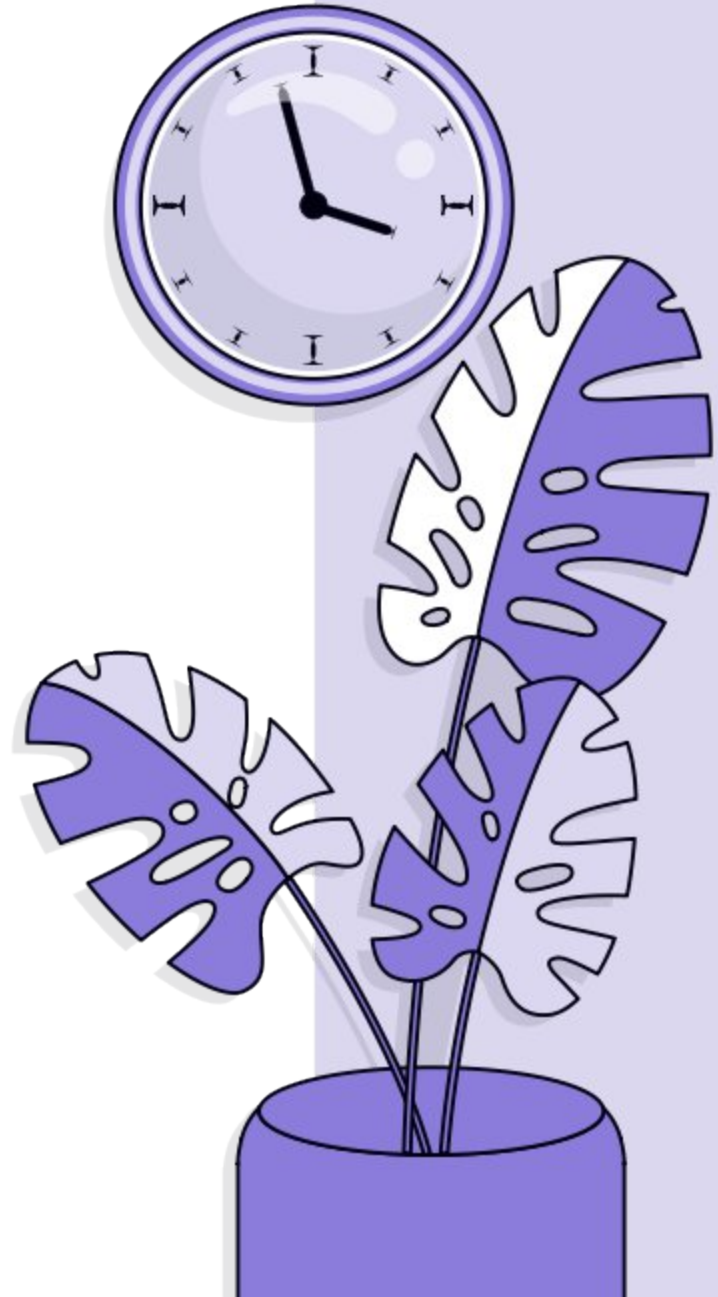
Segurança e Criptografia

Autenticação, HTTPS e SSL/TLS

Por: Ayuka Henrique

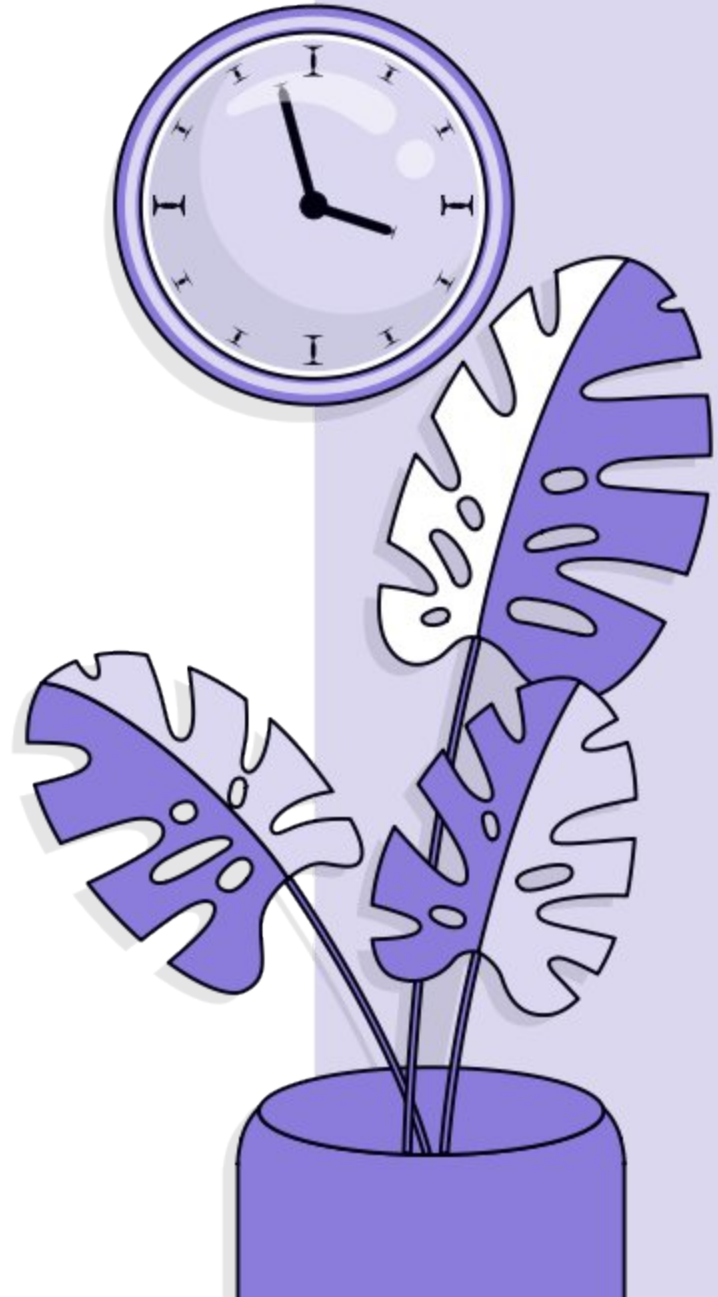
O que vamos ver

- **Autenticação:** Basic Auth, Bearer Tokens (JWT) e OAuth2
- **HTTPS:** A diferença entre tráfego aberto e cifrado
- **SSL/TLS:** Handshake e Autoridades Certificadoras (CA)



Autenticação em APIs

- Processo de verificar **quem** está fazendo a requisição
- Três abordagens comuns:
 - **Basic Auth** — credenciais em toda requisição
 - **Bearer Token (JWT)** — token auto-contido e assinado
 - **OAuth2** — framework de delegação de acesso
- Todas **exigem HTTPS** para serem seguras



Basic Auth

Como funciona:

- Cliente envia Authorization: Basic base64(username:password)
- **Base64 NÃO é criptografia** — é decodificado em milissegundos
- Enviado em **toda requisição** → aparece em logs do servidor

Riscos:

- Só seguro sobre HTTPS (nunca usar em produção pública)
- Sem escopo, sem revogação granular
- Uso aceitável: ferramentas internas, ambiente dev, legacy

Bearer Tokens — JWT (JSON Web Token)

Estrutura do JWT:

- **Header** — algoritmo (ex: RS256)
- **Payload** — claims (sub, exp, iss, aud)
- **Signature** — assinatura criptográfica

Como funciona:

- Servidor valida a assinatura **sem consultar banco de dados**
- **Stateless** — ideal para microsserviços

- **Vantagens:** Stateless, escopo embutido, renovável
- **Desvantagens:** Não revogável sem blocklist, token grande (500b–4KB)

OAuth 2.0 — Delegação de Acesso

O que é:

- **Protocolo de autorização** (não autenticação)
- Permite que um app acesse recursos de um usuário **sem compartilhar a senha**

Fluxo principal — Authorization Code + PKCE:

1. App redireciona usuário ao servidor de autorização
2. Usuário faz login e autoriza o escopo
3. Servidor retorna um code
4. App troca code + code_verifier por access_token
5. Usa o token para acessar a API

Comparação entre os Métodos

Método	Segurança	Complexidade	Stateless	Revogável	Melhor para
Basic Auth	Baixa	Muito Baixa	Sim	Sim (troca senha)	Dev/Interno
JWT (Bearer)	Média	Média	Sim	Difícil	APIs, microsserviços
OAuth2	Alta	Alta	Variável	Sim	Delegação 3rd-party

HTTP — O Tráfego em Texto Aberto

Dados trafegam em **plaintext**

Qualquer um na rede pode **ler e modificar** pacotes

Vulnerável a:

- **Eavesdropping** (espionagem)
- **Man-in-the-Middle** (MITM)
- **Injeção de conteúdo** (malware, anúncios)

▼ Hypertext Transfer Protocol

▶ GET /secret/ HTTP/1.1\r\n

Host: 172.16.1.202\r\n

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:31.0) Gecko/2

Accept: text/html,application/xhtml+xml,application/xml;q=0.

Accept-Language: en-US,en;q=0.5\r\n

Accept-Encoding: gzip, deflate\r\n

Connection: keep-alive\r\n

If-Modified-Since: Thu, 16 Jun 2016 19:05:06 GMT\r\n

If-None-Match: "68-53569ea88b7f7-gzip"\r\n

▼ Authorization: Basic WU9VUi10QU1FOnNlY3JldHBhc3N3b3Jk\r\n

Credentials: YOUR-NAME:secretpassword

\r\n

HTTPS — A Camada de Segurança

HTTP sobre TLS/SSL (porta 443)

Toda comunicação é **criptografada**

Três garantias:

- **Confidencialidade** — só quem tem a chave lê
- **Integridade** — dados não são alterados sem detecção
- **Autenticação** — servidor prova quem é (via certificado)

```
▼ Secure Sockets Layer
  ▼ TLSv1 Record Layer: Handshake Protocol: Client Hello
    Content Type: Handshake (22)
    Version: TLS 1.0 (0x0301)
    Length: 229
    ▼ Handshake Protocol: Client Hello
      Handshake Type: Client Hello (1)
      Length: 225
      Version: TLS 1.2 (0x0303)
      > Random: f4e5aa91712540053d080909719269b21b1d7c0890969a02...
      Session ID Length: 0
      Cipher Suites Length: 40
      > Cipher Suites (20 suites)
      Compression Methods Length: 1
      > Compression Methods (1 method)
      Extensions Length: 144
      > Extension: renegotiation info (len=1)
      ▼ Extension: server_name (len=20)
        Type: server_name (0)
        Length: 20
        ▼ Server Name Indication extension
          Server Name list length: 18
          Server Name Type: host_name (0)
          Server Name length: 15
          Server Name: play.google.com
      > Extension: extended_master_secret (len=0)
      > Extension: signature_algorithms (len=20)
      > Extension: status_request (len=5)
      > Extension: next_protocol_negotiation (len=0)
      > Extension: signed_certificate_timestamp (len=0)
```

HTTP × HTTPS

Característica	HTTP	HTTPS
Porta	80	443
Criptografia	Nenhuma	TLS/SSL
Autenticação	Não	Certificado
Integridade	Não	Sim
Confiabilidade	Baixa	Alta
Performance	Rápido (sem overhead)	1 RTT extra (TLS 1.3)

"HTTPS deve ser usado para todo site, independentemente do tipo de informação que esteja sendo enviada." — Cloudflare

SSL/TLS — Transport Layer Security

O que é:

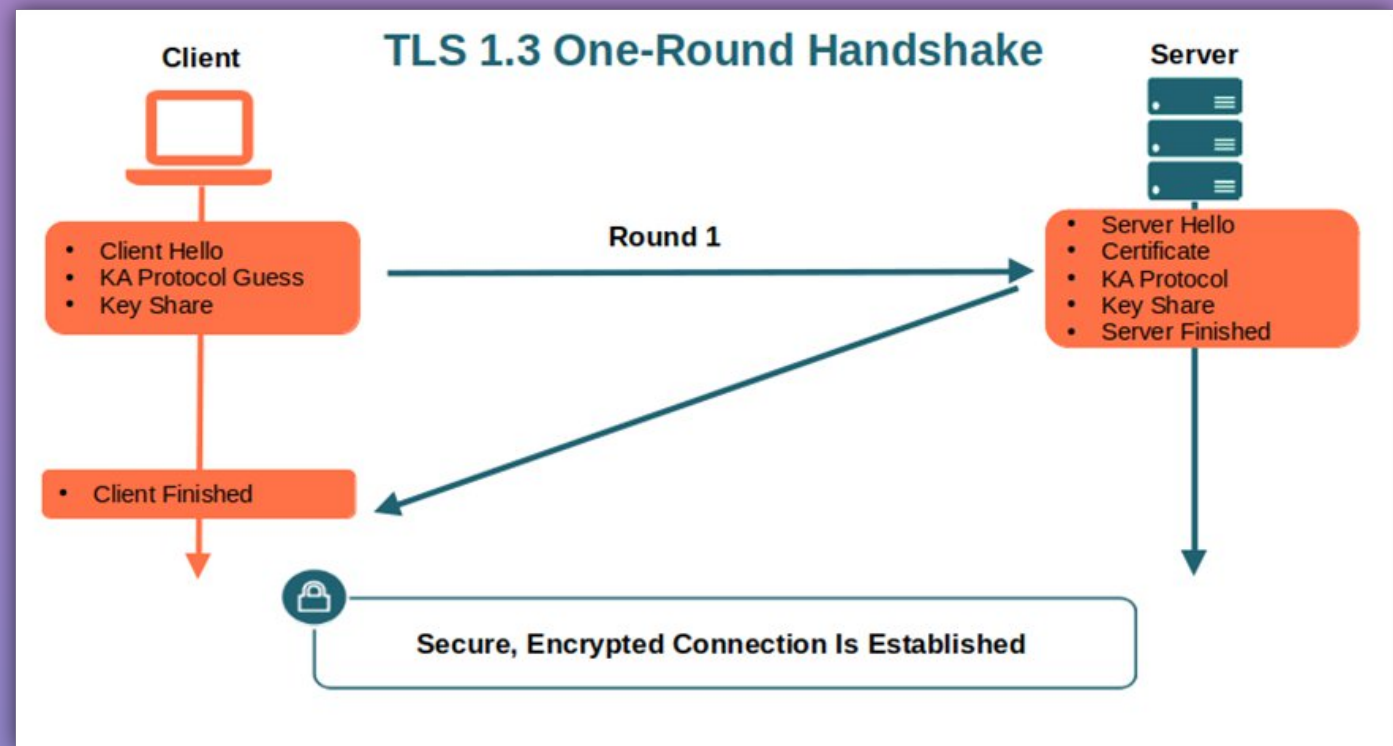
- Protocolo criptográfico que opera entre a camada de transporte (**TCP**) e a aplicação (**HTTP**)
- **SSL** (Secure Sockets Layer) → antecessor (depreciado)
- **TLS** (Transport Layer Security) → versão atual (1.2 e 1.3)

O que faz:

- **Autenticação** — servidor prova identidade via certificado X.509
- **Troca de chaves** — acordo sobre chave de sessão
- **Cifração simétrica** — dados trafegam cifrados (mais rápido)

Handshake TLS 1.3 — Aperto de Mão (1 RTT)

Forward Secrecy: Mesmo que a chave privada do servidor vaze no futuro, sessões passadas **não** podem ser descriptografadas.



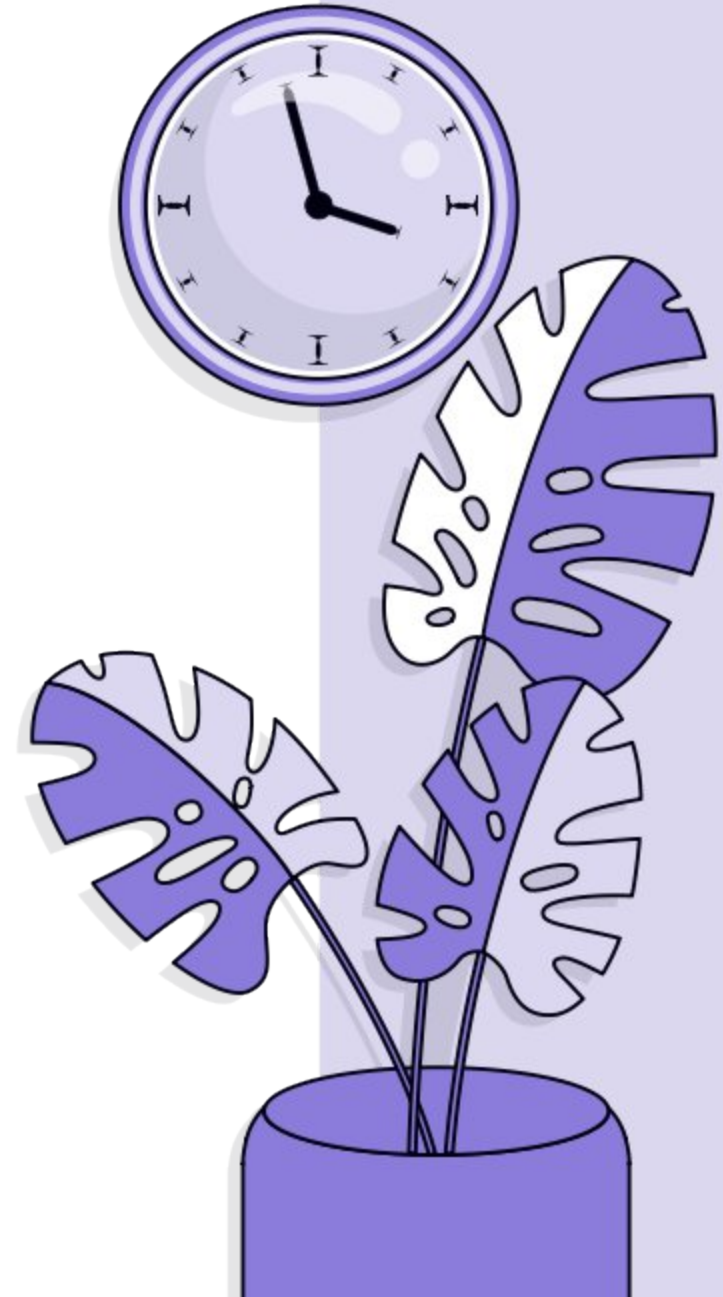
Certificate Authorities (CA)

O que são:

- Terceiros confiáveis que **emitem e assinam** certificados digitais
- Exemplos: Let's Encrypt, DigiCert, GlobalSign, IdenTrust
- Raízes (root CAs) são incorporadas nos navegadores/sistemas operacionais (~200 globalmente)

O que a CA verifica:

Tipo	Verificação	Uso
DV (Domain Validation)	Controle do domínio	Sites comuns
OV (Organization Validation)	Identidade da organização	Empresas
EV (Extended Validation)	Verificação rigorosa	Bancos/governo



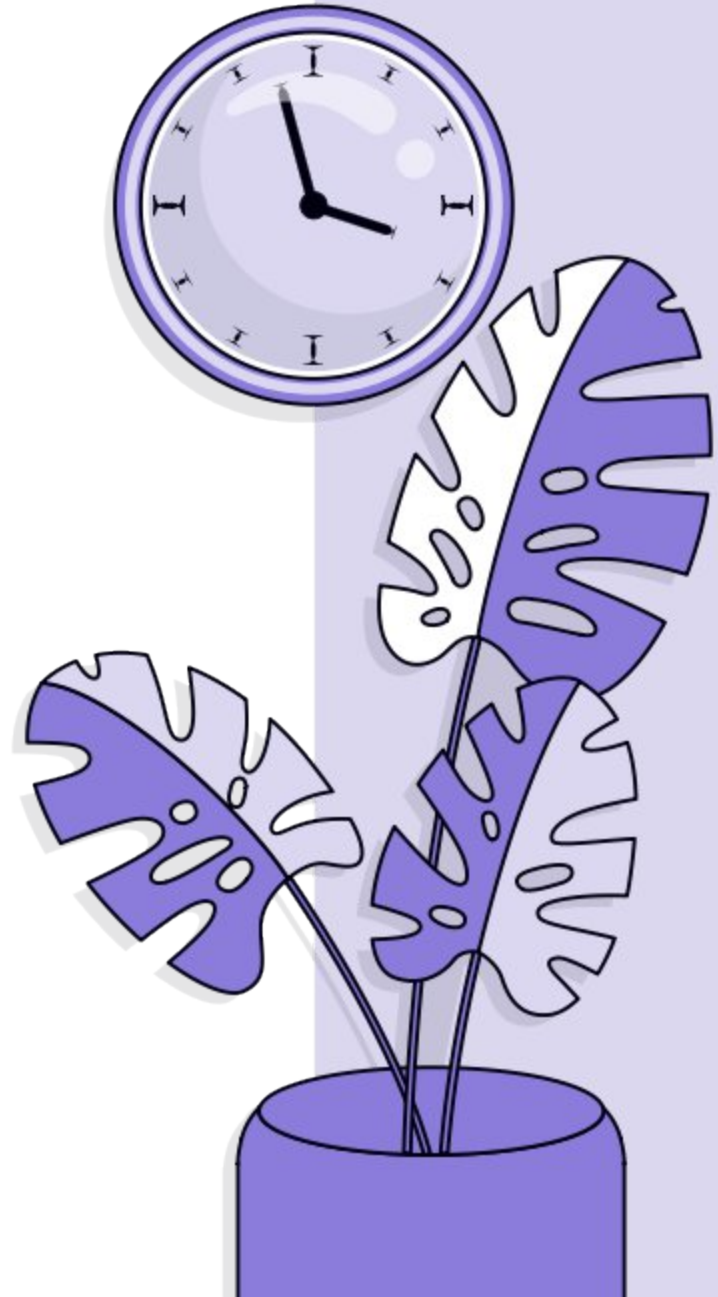
Certificate Authorities (CA)

Cadeia de Confiança:

Root CA (trust store) → Intermediate CA → Seu Certificado (ex: exemplo.com)

Processo de verificação do cliente:

1. Certificado é assinado pela Intermediate → válido
2. Intermediate é assinada pela Root → válida
3. Root está no trust store do navegador → confiável





Por que as CAs são essenciais?

Sem CAs: qualquer um poderia criar um certificado falso para seubanco.com e você nunca saberia a diferença.

O que acontece se a verificação falha:

- Certificado expirado
- Domínio não corresponde ao SAN
- CA emissora não é confiável
- Certificado auto-assinado (sem CA)
- O navegador bloqueia a conexão com erro

Revogação (quando um certificado é comprometido):

- **CRL** (Certificate Revocation List) — lista pública de certificados revogados
- **OCSP** (Online Certificate Status Protocol) — consulta em tempo real
- **OCSP Stapling** — servidor já traz a resposta OCSP junto no handshake



Resumo Final

Camada	Tecnologia	Propósito
Autenticação	Basic / JWT / OAuth2	Quem está fazendo a requisição
Transporte	HTTPS (HTTP + TLS)	Dados cifrados em trânsito
Infraestrutura	TLS / CAs	Confiança e identidade do servidor

3 lições principais:

- 1. Autenticação** — Escolha o método certo: Basic (dev), JWT (APIs), OAuth2 (delegação)
- 2. HTTPS** — Nunca use HTTP em produção; todo dado viaja cifrado
- 3. CAs** — São a base de confiança da web; sem elas, não há como verificar identidade

Referências

Cloudflare Learning Center (SSL/TLS, HTTPS)

botneve.com — API Authentication Patterns (2026)

FreeDevTool Guide — API Authentication Guide (2026)

Zuplo Learning Center — Top 7 API Authentication Methods (2026)

kindatechnical.com — TLS/SSL Deep Dive

routeharden.com — TLS 1.3 Handshake Byte by Byte

Wikipedia / AWS / MDN / RFC 2818

